EXPERIMENT 1

TITLE

8-Puzzle Problem using Breadth-First Search (BFS)

AIM

To write a Python program to solve the **8-Puzzle Problem** using the **Breadth-First Search (BFS)** algorithm.

OBJECTIVE

- To understand the working of the 8-Puzzle problem.
- To implement the Breadth-First Search (BFS) algorithm.
- To find the sequence of moves from the initial state to the goal state.
- To demonstrate state space search in Artificial Intelligence.

ALGORITHM

Step 1: Define the initial state of the puzzle.

Step 2: Define the goal state.

Step 3: Create an empty queue and insert the initial state.

Step 4: Create a visited list to avoid revisiting states.

Step 5: Remove the first state from the queue.

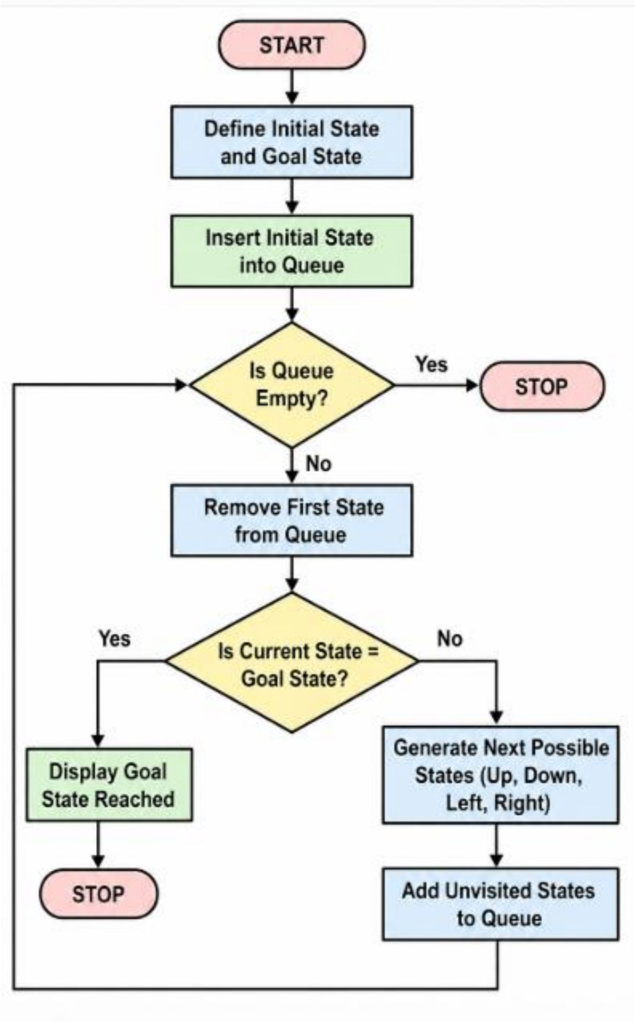
Step 6: If the current state is the goal state, stop.

Step 7: Otherwise, generate all possible next states by moving the blank tile (Up, Down, Left, Right).

Step 8: Add the unvisited states to the queue.

Step 9: Repeat Steps 5–8 until the goal state is found.

FLOWCHART



PYTHON CODE

```
from collections import deque

goal = [1, 2, 3,
        4, 5, 6,
        7, 8, 0]

initial = [1, 2, 3,
           4, 5, 6,
           0, 7, 8]

def get_moves(state):
    moves = []
    pos = state.index(0)

    row = pos // 3
    col = pos % 3

    directions = [(-1,0),(1,0),(0,-1),(0,1)]

    for dr, dc in directions:
        r = row + dr
```

```
        c = col + dc

        if 0 <= r < 3 and 0 <= c < 3:
            new = state[:]
            new_pos = r*3 + c
            new[pos], new[new_pos] = new[new_pos], new[pos]
            moves.append(new)

    return moves

queue = deque()
queue.append((initial, []))

visited = set()

while queue:
    state, path = queue.popleft()

    if tuple(state) in visited:
        continue

    visited.add(tuple(state))

    if state == goal:
        print("Goal State Reached!")
        print("Number of Moves:", len(path))
        print(state)
        break

    for move in get_moves(state):
        queue.append((move, path + [move]))
```

SAMPLE OUTPUT

```
Goal State Reached!
Number of Moves: 2

[1, 2, 3,
 4, 5, 6,
 7, 8, 0]
```

CONCLUSION

The **8-Puzzle Problem** was solved successfully using the **Breadth-First Search (BFS)** algorithm. The algorithm explored all possible states level by level and found the shortest path from the initial state to the goal state.